

RAPPORT 2 — Vision par Ordinateur avec CNN : De l'Implémentation From Scratch à l'Étude Ablative

Projet : Deep Learning — Partie 2 Date : Juin 2026

1. Introduction et Objectifs

Ce rapport explore en profondeur les **Réseaux de Neurones Convolutionnels (CNN)** appliqués à la classification d'images du dataset **Fashion-MNIST**. Au-delà de la simple utilisation de PyTorch, ce travail inclut une **implémentation manuelle (from scratch)** des opérations fondamentales et une **étude ablative** de 5 variantes architecturales distinctes.

Objectifs principaux :

- 1. Comprendre intimement les mécanismes internes des CNN (corrélation, pooling).
- 2. Comparer les performances structurelles CNN vs MLP sur des données à forte cohérence spatiale.
- 3. Évaluer précisément l'impact de chaque choix architectural sur la convergence globale.

2. Méthodologie

2.1 Dataset Fashion-MNIST

Caractéristique	Valeur
Images	70,000 (60k train + 10k test)
Résolution	28×28 pixels, 1 canal (niveaux de gris)
Classes	10 catégories (T-shirt, Trouser, Pullover, Dress, Coat, Sandal, Shirt, Sneaker, Bag, Ankle boot)
Normalisation	mean=0.2860, std=0.3530
Split	85% train / 15% validation

2.2 Implémentation From Scratch

Corrélation 2D Manuelle

```
for i in range(H_out):
    for j in range(W_out):
        window = input[:, :, i:i+kH, j:j+kW]
        output[:, :, i, j] = (window.unsqueeze(1) * kernel).sum(dim=(2,3,4))
```

Validation : Les résultats obtenus sont strictement identiques à ceux de `nn.Conv2d` (avec une tolérance numérique de 1e-6) .

Max-Pooling et Average-Pooling Manuels

- **Max-Pooling** : préserve les features les plus actives et offre une invariance aux petites translations.
- **Average-Pooling** : lisse uniformément les features, atténuant le bruit spatial.

Validation : Résultats rigoureusement validés contre `F.max_pool2d` et `F.avg_pool2d` .

Détection de Bords (Filtre Sobel)

Application d'un filtre de Sobel manuel sur une sélection d'images Fashion-MNIST, validée avec succès via l'utilisation détournée de `F.conv2d`.

2.3 Architectures Comparées

LeNet (CNN de référence)

```
Conv2d(1→6, k=5) → ReLU → MaxPool(2) → Conv2d(6→16, k=5) → ReLU → MaxPool(2) → Flatten
```

Nombre total de paramètres : 44,426

SimpleMLP (Baseline comparative)

```
Flatten → Linear(784→128) → ReLU → Linear(128→64) → ReLU → Linear(64→10)
```

Nombre total de paramètres : 109,386

2.4 Étude Ablative : 5 Variantes

Variante	Modification	Hypothèse sous-jacente
V1	Padding = 2	Préserver au maximum la résolution spatiale aux bords.
V2	Stride = 2 (sans pooling)	Remplacer le pooling traditionnel par une convolution stridée.
V3	Average pooling	Évaluer l'effet du lissage global vs la sélection maximale.
V4	Plus de filtres (16→32)	Augmenter significativement la capacité de représentation.
V5	Convolutions 1×1	Opérer une réduction de dimension de canaux à faible coût.

3. Résultats Expérimentaux

3.1 Comparaison MLP vs CNN (Entraînement : 5 epochs)

Modèle	Paramètres	Val Accuracy	Gain paramétrique
SimpleMLP	109,386	87.61%	Baseline
LeNet (CNN)	44,426	87.66%	~2.5× moins de paramètres

Constat majeur : Le CNN atteint une performance équivalente, voire légèrement supérieure, tout en mobilisant 2.5 fois moins de paramètres, grâce au mécanisme de partage des poids.

3.2 Étude Ablative Fine (Entraînement restreint : 1 epoch)

Modèle / Variante	Val Accuracy	Paramètres	Observation critique
LeNet (baseline)	81.99%	44,426	Valeur de référence.
Avec padding (V1)	82.98%	107,786	+1% d'accuracy mais explosion des paramètres (+143%).
Stride=2 (V2)	80.97%	44,426	Perte d'information spatiale fine.
Average pooling (V3)	76.04%	44,426	Trop de lissage, baisse drastique ❌.
Plus de filtres (V4)	83.98%	85,822	Meilleur compromis performance/capacité ✅.
Conv 1×1 (V5)	76.66%	43,654	Réduction de dimensionnalité trop agressive.

3.3 Visualisation des Feature Maps

- **Première Couche (6 filtres)** : Spécialisation claire dans la détection de contours abrupts et de textures simples.
- **Seconde Couche (16 filtres)** : Combinaison de ces premiers contours en motifs géométriques et structures plus complexes (manches, talons, cols).

4. Analyse et Discussion

4.1 Pourquoi le CNN s'avère-t-il structurellement plus efficace ?

1. **Partage des poids** : Un filtre unique balaie l'intégralité de l'image, réduisant drastiquement le nombre de connexions synaptiques par rapport à un MLP.
2. **Invariance translationnelle** : Un motif localisé en haut à gauche est parfaitement reconnu s'il se déplace en bas à droite.
3. **Hierarchisation des features** : Les couches profondes synthétisent les informations élémentaires des premières couches en concepts sémantiques riches.

4.2 Analyse Détaillée des Variantes Ablatives

Average Pooling vs Max Pooling : La chute brutale de performance (-6%) met en lumière le fait que le max-pooling capture les activations maximales (les éléments les plus discriminants de l'image) tandis que l'average-pooling dilue l'information utile en la moyennant avec le bruit de fond.

Plus de filtres : L'augmentation contrôlée de la capacité (16 vers 32 filtres) offre une meilleure granularité de capture des motifs complexes, validée par une hausse notable de l'accuracy (83.98%).

5. Conclusion

Cette suite d'expériences valide la **supériorité structurelle incontestable** des CNN sur les architectures purement denses (MLP) pour le traitement de données matricielles :

- **Efficacité paramétrique extrême** : Moins de ressources pour un résultat équivalent.
- **Max-pooling > Average-pooling** pour les tâches de classification pure.
- L'implémentation **from scratch** permet de valider mathématiquement et de maîtriser les couches d'abstractions PyTorch couramment utilisées.

6. Références

- LeCun et al. (1998). *Gradient-based learning applied to document recognition*
- Simonyan & Zisserman (2014). *Very Deep Convolutional Networks*
- Documentation PyTorch : `torch.nn.Conv2d` , `torch.nn.MaxPool2d`